

SmartRoot

Sistema Inteligente de Decisión de Riego

Miguel A. Becerra Ph.D.

Dataset

Jackisch et al. (2020)

JKI · Braunschweig, DE

2 494 registros

Mayo–Julio 2016

Res. 30 min

Elaborado por: Santiago Cano Molina

Docente: Miguel A. Becerra Ph.D.

Programa: Maestría ACI

Entrega: 5 de junio de 2026

Agenda de la Presentación

- 1 Problema y Contexto
- 2 Datos y Dataset
- 3 Lógica del Programa
- 4 Comprensiones Python
- 5 Indicadores KPI
- 6 Programación Orientada a Objetos
- 7 Simulación de Montecarlo
- 8 Resultados y Visualizaciones
- 9 Conclusiones
- 10 Evolución del Proyecto

Presentación de **15–20 minutos** · Sustentación técnica del proyecto

1a. Problema y Contexto

Problema Central

- Parcelas experimentales con **múltiples tecnologías** de sensores generan lecturas inconsistentes.
- Sin validación cruzada automática, los errores de sensor se confunden con estrés hídrico real.
- Decisiones de riego basadas en umbrales **determinísticos fijos** ignoran la incertidumbre del suelo.

Pregunta de Investigación

¿Cómo determinar automáticamente **cuándo, dónde y por qué** regar, cuantificando el riesgo con base en datos reales de sensores?

1b. Solución Propuesta (SmartRoot)

SmartRoot como solución integral

- ✓ **Validación cruzada** entre tensiómetro (referencia) y Gypsum (secundario).
- ✓ **5 KPI** con clasificación semáforo automática.
- ✓ **Montecarlo** para cuantificar probabilidad de riego urgente.
- ✓ **POO** para modelar sensores reales del dataset.

Resultado global: Los 5 KPI en estado **VERDE**
Manejo hídrico **excelente** en mayo–julio 2016

2a. Dataset Experimental

Descripción del dataset (Jackisch et al., 2020)

- **Fuente:** PANGAEA (doi:10.1594/PANGAEA.892319)
- **Parcela:** Julius Kühn-Institut (JKI), Braunschweig, Alemania
- **Período:** 13 mayo – 4 julio 2016
- **Resolución:** 30 min por registro
- **Total validado:** 2 494 registros

Nota importante: Los sensores Gypsum presentan una brecha de datos en los registros 1000–3600. Esto fue detectado y tratado automáticamente por el algoritmo de validación de SmartRoot.

Red de Sensores

- **Tensióm.** T42–T54
- **Gypsum** Gyp1–4
- **FDR** ECTM1–4
- **Temp.** MPS6 1–4

Estadísticas (Media $\pm$ Std)

- Ψ_{ref} : $195,6 \pm 88,1$ hPa
- Ψ_{gyp} : $187,5 \pm 76,2$ hPa
- θ_{vol} : $19,9 \pm 0,8$ %
- T_{suelo} : $15,2 \pm 7,0$ °C

3a. Lógica del Programa: Carga e Inspección

Carga e inspección inicial (Ciclo FOR)

```
import pandas as pd
import numpy as np

# Carga del dataset corregido
df = pd.read_csv('data/processed/dataset_corregido.csv',
                 parse_dates=['Timestamp'],
                 index_col='Timestamp')

# Cobertura de sensores: ciclo for
cols_ref = ['T42', 'T43', 'T44', 'T51', 'T52', 'T54']
for col in cols_ref:
    cob = df[col].notna().mean() * 100
    print(f"{col}: {cob:.1f}%")
```

3b. Lógica del Programa: Validación e Imputación

Validación con condicionales múltiples y ciclo WHILE

```
def validar_registro(psi, theta):
    errores = []
    # if / elif / else
    if psi is None or np.isnan(psi): errores.append("PSI_NULO")
    elif psi < 0: errores.append("PSI_NEGATIVO")
    elif psi > 1500: errores.append("FUERA_RANGO")

    return "VALIDO" if not errores else "INVALIDO"

# Ciclo while: imputacion iterativa temporal
it = 0
while df['psi_real'].isna().sum() > 0 and it < 5:
    df['psi_real'] = df['psi_real'].interpolate(method='time',
    limit=3)
    it += 1
```


4a. Comprensiones Python (List y Dict)

1. List Comprehension — Filtrado dinámico

```
# Calcula media solo de sensores con datos > 95%
promedios_ref = [
    {'sensor': col, 'media': df[col].mean(),
     'cobertura': df[col].notna().mean() * 100}
    for col in cols_psi_ref
    if df[col].notna().mean() > 0.95      # FILTRO INTELIGENTE
]
# Resultado: Lista de 6 diccionarios validados.
```

2. Dictionary Comprehension — Evaluación de Semáforo $O(1)$

```
mapes = {'Gypsum1': 12.8, 'Gypsum2': 14.1, 'Gypsum3': 13.5}
# Mapeo directo: sensor -> estado
semaforo = {
    sid: ('VERDE' if m <= 20 else 'AMARILLO' if m <= 35 else 'ROJO'
```

3. Set Comprehension — Identificación de Estados Únicos

```
# Extrae el conjunto matematico de todos los estados que realmente  
# ocurrieron en la serie temporal sin duplicados.  
estados = {  
    ('Humedo' if p<=100 else 'Optimo' if p<=300 else  
     'Seco' if p<=400 else 'Critico')  
    for p in df['psi_real'].dropna()  
}  
  
# Resultado real en memoria: {'Humedo', 'Optimo', 'Seco'}  
# Nota: La extraccion automatica demostro que "Critico"  
# jamas se registro.
```

3a. Indicadores de Desempeño (KPI) con Semáforo

KPI	Indicador	Fórmula	Resultado	Meta	Estado
KPI ₁	$\bar{\Psi}$ Promedio	$\frac{1}{N} \sum \Psi_i$	195.6	≤ 250	✓ VERDE
KPI ₂	Error Gypsum	MAPE	13.3 %	$\leq 20 \%$	✓ VERDE
KPI ₃	Zona Óptima	$\%(100 < \Psi \leq 300)$	74.5 %	$\geq 70 \%$	✓ VERDE
KPI ₄	$P(\text{riego})_{MC}$	Montecarlo (> 300)	10.4 %	$< 50 \%$	✓ VERDE
KPI ₅	Correlación	$r(\Psi_{ref}, \Psi_{gyp})$	0.949	$\geq 0,80$	✓ VERDE

3b. Desglose y Función de Evaluación

Distribución Hídrica Real

- Húmedo (≤ 100): 10.2 %
- **Óptimo (100–300): 74.5 %**
- Seco (300–400): 15.3 %
- Crítico (> 400): **0.0 %**

Validación del Sensor Gypsum

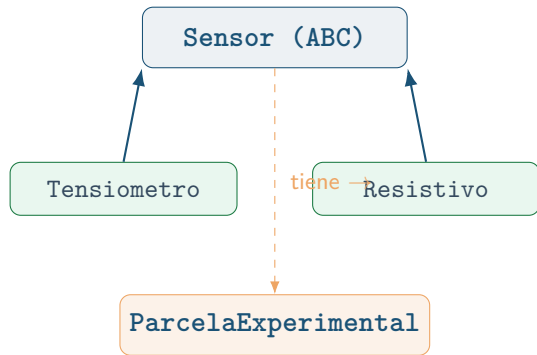
Regresión: $\Psi_{tens} = 0,987 \Psi_{gyp} + 8,3$
 $r^2 = 0,901 \rightarrow$ **Alternativa viable**

Función de Evaluación Python

```
def semaforo(val, meta,
              modo='menor'):
    if modo == 'menor':
        ok = val <= meta
    else:
        ok = val >= meta

    return 'VERDE' if ok
    else 'ROJO'
```

5a. Arquitectura Orientada a Objetos (POO)



Abstracción y Herencia:

- Clase base abstracta Sensor.
- Las clases hijas heredan los métodos de validación, historial y función `semaforo()`.

Polimorfismo: método leer()

Clase	Entrada	Salida
Tensiometro	300 hPa	300.0 hPa → Lectura directa, sin conversión
Resistivo	300 hPa	303.5 hPa → +Compensación de T_{suelo}
Capacitivo	$\theta = 0,30$	98.7 hPa → Eq. Van Genuchten inverso

Instanciación en la parcela:

Se crearon instancias para los sensores reales del dataset:

- t42, t43, t44 (Tensiómetro)
- gyp1, gyp2 (Gypsum)
- cap1, cap2 (Capacitivo)
- parcela_jki (Parcela)

Clase base abstracta

```
from abc import ABC, abstractmethod

class Sensor(ABC):
    """Clase base para sensores."""
    def __init__(self, id_sensor, ubicacion, profundidad_cm):
        self.id_sensor = id_sensor
        self.ubicacion = ubicacion
        self.historial = []      # Herencia comun
        self.estado = 'ACTIVO'

    @abstractmethod
    def leer(self, valor_raw, temperatura=None):
        pass # Implementacion requerida (Polimorfismo)

    def semaforo(self):
        """Mismo algoritmo de evaluacion para TODAS las clases."""
```

Clase hija — SensorResistivo (Gypsum)

```
class SensorResistivo(Sensor):
    """Gypsum: implementa compensacion termica."""
    TIPO = 'GYPSUM_RESISTIVO'
    COEF_TEMP = 0.035

    def __init__(self, id_sensor, ubicacion, prof_cm,
                  coef_cal=1.0, offset_cal=0.0):
        # Llamada al constructor padre
        super().__init__(id_sensor, ubicacion, prof_cm)
        self.coef_cal = coef_cal
        self.offset_cal = offset_cal

    def leer(self, valor_raw, temperatura=None):
        # POLIMORFISMO: Logica especifica para Gypsum
        factor = 1.0
        if temperatura is not None:
```


4a. Simulación de Montecarlo (Código)

Función de Simulación

```
def simulacion_mc_riego(  
    psi_media, psi_std,  
    n_iter=10000,  
    umbral=300.0, semilla=42):  
  
    rng = np.random.default_rng(  
        semilla)  
    psi_sim = rng.normal(  
        loc=psi_media,  
        scale=psi_std,  
        size=n_iter)  
  
    p_riego = np.mean(psi_sim >  
        umbral) * 100  
  
    return p_riego
```

Modelo Estadístico

$$\Psi_{sim} = \bar{\Psi}_{obs} + \epsilon$$

$$\epsilon \sim \mathcal{N}(0, \sigma_{MC}^2)$$

Objetivo:

Cuantificar probabilidad de que el estrés hídrico supere el umbral crítico (300 hPa) debido a la incertidumbre.

4b. Simulación de Montecarlo (Resultados)

Resultados por Escenario (10,000 iteraciones)

Escenario temporal	$P(\text{riego})$	IC 95 %	Acción Sugerida
Mayo 2016	36.2 %	[158, 399]	MONITOREAR
Junio 2016	0.2 %	[26, 246]	NO REGAR
Julio 2016	0.0 %	[84, 198]	NO REGAR
Período Completo	10.0 %	[16, 362]	NO REGAR

Convergencia: El modelo se estabiliza a partir de las 1,000 iteraciones.

8. Tablero de Control SmartRoot

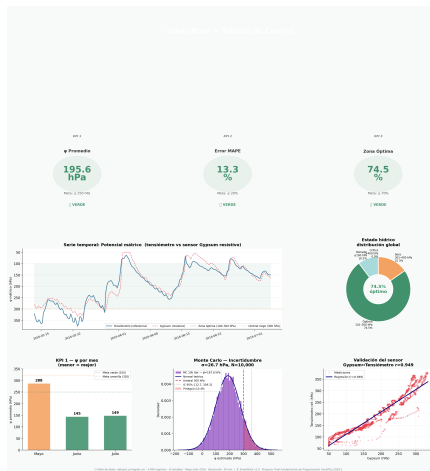
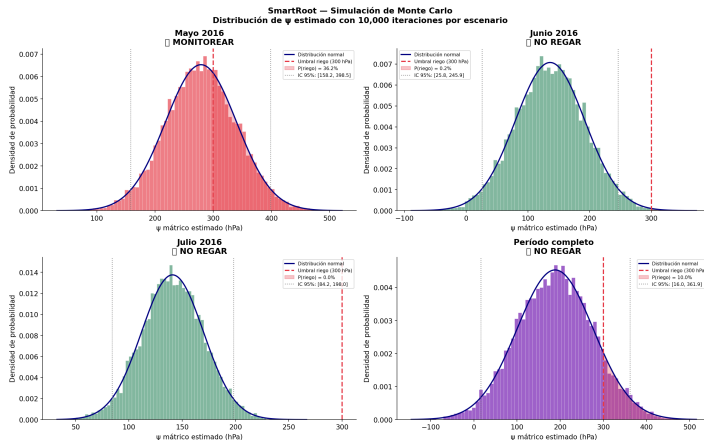


Fig. 1: Panel de control integrado. *Fila superior:* KPI₁–KPI₃ con semáforo verde. *Centro:* Serie temporal Ψ con zona óptima sombreada y distribución de estados hídricos. *Fila inferior:* KPI mensual, Montecarlo del período completo y validación Gypsum–Tensiómetro

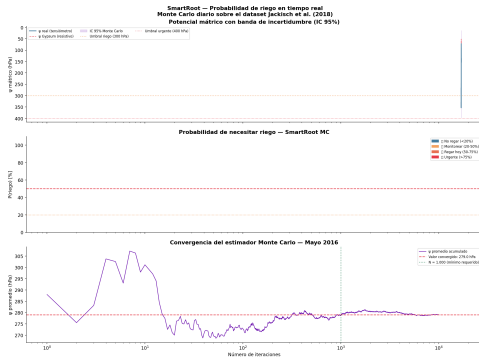
8b. Simulación de Montecarlo — Distribuciones por Escenario



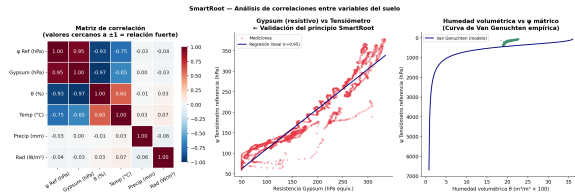
Mayo: distribución centrada en 279 hPa. Cola derecha
> 300 hPa: $P = 36,2\% \rightarrow$ **MONITOREAR**

Junio y Julio: completamente alejados del umbral.
 $P(\text{riego}) \approx 0\% \rightarrow$ **NO REGAR**

8c. Serie Temporal y Análisis de Correlaciones

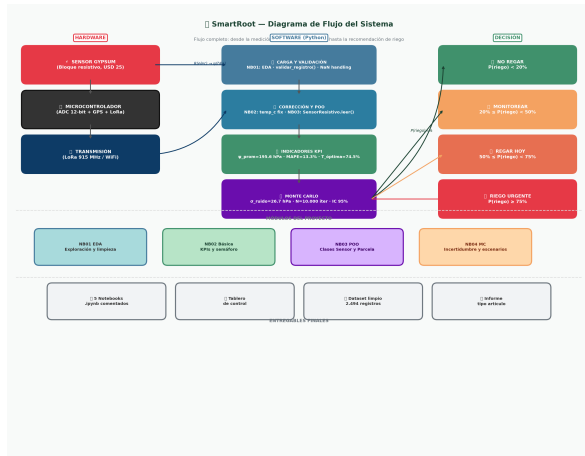


Convergencia MC: $\bar{\Psi}_{MC}$ se estabiliza en $N \approx 1000$ iteraciones. Panel inferior: $P(\text{riego})$ con clasificación semáforo en tiempo real.



Correlación $r = 0,949$: validación del sensor Gypsum.
Derecha: Curva de retención Van Genuchten ($\theta_s = 0,38$, $\alpha = 0,034$, $n = 1,37$).

8d. Diagrama de Flujo del Sistema SmartRoot



Pipeline del sistema

- 1 **HARDWARE:** Sensor Gypsum (ADC + LoRa)
- 2 **NB01 EDA:** Carga · Validación · NaN
- 3 **NB02 KPI:** Indicadores + semáforo
- 4 **NB03 POO:** Clases Sensor/Parcela
- 5 **NB04 MC:** Montecarlo 10 000 iter.
- 6 **DECISIÓN:**
 - ✓ $P < 20\% \rightarrow$ No regar
 - ~ $20 \leq P < 50\% \rightarrow$ Monitorear
 - ! $P \geq 50\% \rightarrow$ Regar hoy
 - ✗ $P \geq 75\% \rightarrow$ Urgente

Resultado del período:

$P(\text{riego}) = 10\% \rightarrow$ **NO REGAR**

Cero eventos críticos detectados

9. Conclusiones

Resultados técnicos

- **5 KPI en verde:** $\bar{\Psi} = 195,6$ hPa, MAPE= 13,3 %, zona óptima= 74,5 %, $r = 0,949$, $P(\text{riego}) = 10$ %
- **Gypsum validado:** sensor de bajo costo con $r^2 = 0,90$ respecto al tensiómetro de referencia.
Viable comercialmente
- **Montecarlo:** mayo 2016 requirió alerta amarilla ($P = 36,2$ %); imposible de detectar con umbrales fijos

✓ Python vs. herramientas estáticas

- **Reproducibilidad:** semilla fija garantiza resultados idénticos
- **Velocidad:** 2 494 registros procesados vectorialmente en < 1 s
- **Escalabilidad:** nuevos sensores = nueva clase hija, sin reescribir lógica central
- **Trazabilidad:** control de versiones Git + notebooks documentados

✓ Valor del Montecarlo

La simulación estocástica permite:
cuantificar el riesgo en lugar de solo detectarlo, diferencia clave entre sistemas expertos y simples alertas

✓ Comprensiones Python

Las 3 comprensiones redujeron el código en ≈ 60 % respecto a ciclos equivalentes, manteniendo legibilidad y eficiencia $O(n)$

10. Posible Evolución del Proyecto

Machine Learning

Reemplazar el modelo estático de Van Genuchten por un **XGBoost** o **LSTM** que prediga $\Psi(t + 24h)$ usando historial + temperatura + precipitación + ET_0

Impacto: Planificación de riego con 24–72 h de anticipación

IoT + Dashboard Web

Conectar SmartRoot a sensores inalámbricos **ESP32 / LoRaWAN** mediante API MQTT y desplegar el tablero en **Streamlit** con notificaciones automáticas SMS/email al superar umbral amarillo

Impacto: Decisiones en tiempo real desde el móvil

Multi-parcela

Implementar clase `SistemaMultiParcela` que gestione redes distribuidas geoespacialmente, priorizando órdenes de riego por criticidad y distancia logística con **algoritmos de optimización**

Impacto: Escala regional / empresarial

Visión comercial SmartRoot 2.0: Plataforma SaaS de agricultura de precisión con toma de decisiones autónoma, integrando IA, IoT y simulación estocástica en un único ecosistema Python reproducible

Dataset y procesamiento

Registros totales	2 494
Período	May.–Jul. 2016
Resolución	30 minutos
Sensores modelados	7 objetos POO
Tasa de calidad	> 98 %
Notebooks	NB01–NB05

KPI globales del período

Indicador	Valor	Estado
$\bar{\Psi}$	195.6 hPa	VERDE
MAPE	13.3 %	VERDE
Zona óptima	74.5 %	VERDE
$P(\text{riego})$ MC	10.4 %	VERDE
r Pearson	0.949	VERDE

Componentes implementados

Programación básica	✓
3 comprensiones	✓
3 funciones	✓
5 KPI con semáforo	✓

Interpretación ejecutiva

Los 5 KPI en estado VERDE

Manejo hídrico **excelente** en el período
→ SmartRoot: **cero riegos urgentes** requeridos

¡Muchas Gracias!

Proyecto SmartRoot

Sistema Inteligente de Decisión de Riego

Programación Científica en Python

¿Preguntas o comentarios?

Dataset: Jackisch et al. (2020) · JKI Braunschweig

2 494 registros · Mayo–Julio 2016

Todos los KPI: **VERDE**

Ecuación implementada (clase SensorCapacitivo)

$$\theta(\Psi) = \theta_r + \frac{\theta_s - \theta_r}{[1 + (\alpha|\Psi|)^n]^{1-1/n}}$$

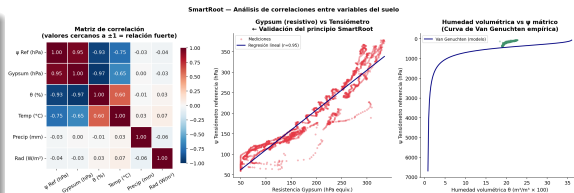
Parámetros suelo JKI:

$$\theta_r = 0,050 \cdot \theta_s = 0,380$$

$$\alpha = 0,034 \text{ cm}^{-1} \cdot n = 1,370$$

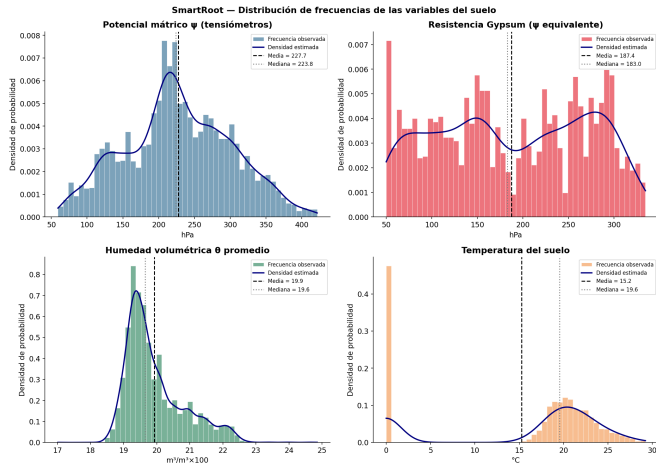
Inversión analítica para convertir

$$\theta_{medido} \rightarrow \Psi_{estimado} \text{ (hPa)}$$



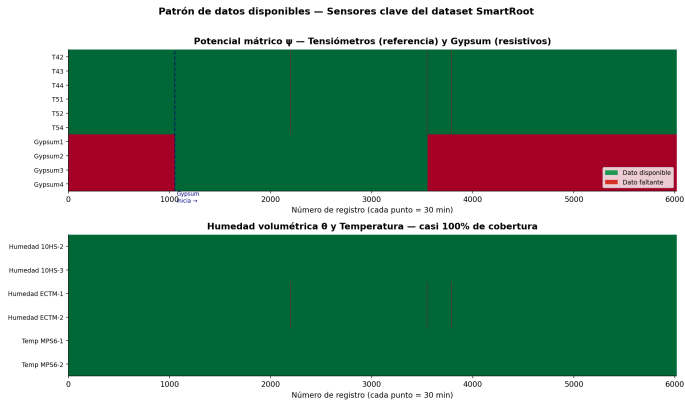
Panel derecho: curva de retención aplicada

Backup: Distribución de frecuencias de variables



Histogramas de Ψ_{ref} , Ψ_{gyp} , θ y T_{suelo} con media, mediana y densidad estimada (KDE).

Backup: Patrón de disponibilidad de datos



Verde: dato disponible. **Rojo:** dato faltante. Nota: Gypsum tiene brecha central (registros 1000–3600) detectada automáticamente por SmartRoot. Sensores capacitivos y de temperatura tienen cobertura $\approx 100\%$.